

SYNOPSIS
OF
AUTONOMOUS OPS/DEV AGENT WITH REAL GUARDRAILS

A Self-Correcting Multi-Agent DevOps System with Live Guardrails & Real-Time Control Dashboard

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted By

Vasu Gaur :- 11242503

Vinayak Sharma :- 11242504

Yadwinder Singh :- 11242691



Under The Supervision of :-

Project Incharge :-

Dr. Vinisha Sumra

Department of Computer science and Engineering

Maharishi Markandeshwar Engineering College

Maharishi Markandeshwar (Deemed to be University)

Mullana, Ambala, Haryana

Academic Session: 2026-27

CHAPTER INDEX

S.No	Chapters	Title
1	Chapter1	Introduction, Problem Statement, Proposed Solution & Scope
2	Chapter2	Objectives and Literature Review
3	Chapter3	Identification of Gaps and Need of Proposed System
4	Chapter4	Proposed Work and System Modules
5	Chapter5	Methodology, Architecture and Requirements
6	Chapter6	Expected Results
7	Chapter7	Conclusion and Future Scope
8	Refrences	Bibliography

CHAPTER 1

Introduction, Problem Statement, Proposed Solution & Scope

1.1 Introduction

Artificial Intelligence has rapidly transformed software development by automating coding, testing, debugging, documentation, and deployment tasks. Modern Large Language Models (LLMs) such as GPT, Claude, Gemini, and LLaMA assist developers in writing code; however, they still require continuous human supervision. Developers are responsible for copying generated code, executing it locally, fixing errors, rerunning tests, and finally deploying the solution.

This manual workflow makes software maintenance time-consuming and inefficient, especially for organizations handling hundreds of bug reports every day. Current AI coding assistants behave like advisors rather than autonomous developers because they cannot independently execute code, validate results, or correct their own mistakes.

The proposed project, **Autonomous Ops/Dev Agent with Real Guardrails**, introduces a fully autonomous multi-agent software engineering system capable of accepting software issues, planning the solution, generating code, executing tests, reviewing its own output, correcting failures automatically, and escalating only when necessary. Unlike traditional AI assistants, the proposed system performs the complete software development lifecycle with minimal human intervention.

The project employs multiple specialized AI agents, each responsible for a dedicated task. These agents collaborate through a structured workflow, enabling reliable and transparent automation. Every decision taken by an agent is recorded in a centralized database, while a real-time dashboard provides live visibility into the execution process.

To ensure safe execution, all generated code runs inside Docker containers, preventing unauthorized access to the host machine. Guardrail mechanisms validate generated outputs before execution, ensuring security, correctness, and compliance.

The system integrates FastAPI, LangGraph, CrewAI, Docker, PostgreSQL, GitHub APIs, WebSockets, and modern AI models to build an intelligent DevOps automation platform.

1.2 Problem Statement

Modern software development teams face several challenges during maintenance and feature implementation.

Some of the major problems include:

- Increasing backlog of GitHub issues.
- Repetitive manual debugging.
- Time-consuming testing process.
- Continuous switching between AI chatbot and IDE.

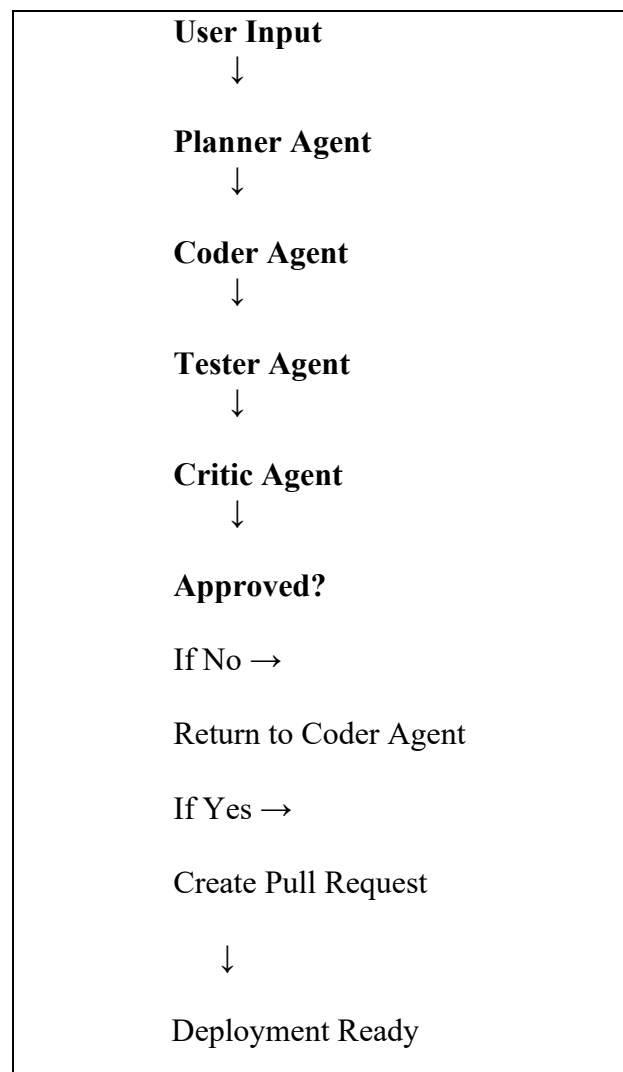
- Human dependency for validation.
- Lack of automatic correction.
- Unsafe execution of AI-generated code.
- Poor transparency during automation.
- No centralized monitoring dashboard.
- Manual deployment workflow.

Although AI assistants can generate code, they cannot autonomously verify whether the generated solution actually works. Developers still perform testing, debugging, and deployment manually.

1.3 Proposed Solution

The proposed solution is an AI-powered autonomous DevOps platform consisting of multiple intelligent agents collaborating together.

The system follows an autonomous execution pipeline:



The major agents are:

- **Planner Agent**

Reads GitHub issues or task descriptions and divides them into structured subtasks.

- **Coder Agent**

Generates implementation code inside an isolated Docker sandbox.

- **Tester Agent**

Executes unit tests, integration tests, and validates generated code.

- **Critic Agent**

Reviews code quality independently and either approves or rejects the implementation.

- **Escalation Agent**

Stops execution whenever confidence falls below predefined thresholds and notifies human developers.

The proposed architecture ensures that software changes are verified before deployment, significantly reducing human effort and improving reliability.

1.4 Scope of the Project

The scope of the project includes development of a complete AI-driven autonomous software engineering platform.

Major functionalities include:

- GitHub Issue Processing
- Automatic Task Planning
- Multi-Agent Coordination
- Autonomous Code Generation
- Docker Sandbox Execution
- Automatic Unit Testing
- Self-Correction Loop
- Live Dashboard Monitoring
- Audit Logging
- Pull Request Generation
- Secure Execution
- Confidence Evaluation
- Human Escalation
- Performance Analytics

The system can initially be deployed locally and later migrated to cloud infrastructure such as AWS, Azure, or Google Cloud Platform.

Potential users include:

- Software Companies
- DevOps Teams
- Open Source Communities
- Enterprise Development Teams
- Educational Institutions
- Freelance Developers

CHAPTER 2

Objectives and Literature Review

2.1 Objectives

The primary objective of this project is to develop a self-correcting autonomous software engineering platform capable of reducing manual intervention during software development.

The major objectives are:

- To develop an autonomous AI software engineering platform.
- To automate issue analysis.
- To automatically generate implementation plans.
- To perform secure code generation.
- To execute generated code inside Docker containers.
- To integrate automated testing.
- To build an independent AI reviewer.
- To implement self-correction without human intervention.
- To provide confidence-based escalation.
- To maintain complete audit logs.
- To develop a real-time monitoring dashboard.
- To integrate GitHub repositories.
- To improve software development productivity.
- To minimize software defects.
- To ensure secure AI execution.

2.2 Literature Review

Artificial Intelligence has significantly influenced software engineering through intelligent code generation and automation. Existing coding assistants such as GitHub Copilot, ChatGPT, Claude, and Gemini provide useful code suggestions; however, they mainly function as interactive assistants requiring continuous user involvement.

Recent advancements in Multi-Agent Systems demonstrate that specialized AI agents can collaboratively solve complex software engineering tasks. Frameworks such as CrewAI and LangGraph enable coordinated communication between multiple autonomous agents.

Containerization technologies like Docker have become standard for secure execution environments, while FastAPI enables scalable backend services. PostgreSQL offers reliable storage for maintaining execution history and audit logs.

The proposed project combines these technologies into a unified autonomous DevOps platform capable of planning, coding, testing, reviewing, correcting, and monitoring software development tasks with minimal human intervention.

CHAPTER 3

Identification of Gaps and Need for Proposed System

3.1 Identified Gaps

Current AI coding assistants exhibit several limitations:

- They only generate code.
- Manual testing is required.
- No automatic debugging.
- No independent code review.
- Lack of execution safety.
- No retry mechanism.
- Limited workflow automation.
- Poor transparency.
- No live monitoring.
- No autonomous deployment pipeline.

3.2 Need of Proposed System

The proposed system addresses these limitations by introducing:

- Autonomous planning.
- Multi-agent collaboration.
- Automatic code execution.
- Continuous testing.
- Independent AI review.
- Self-correction loop.
- Confidence-based escalation.
- Live monitoring dashboard.
- Complete audit trail.
- Secure sandbox execution.

CHAPTER 4

Proposed Work and System Modules

4.1 Proposed Work

The proposed project aims to develop an intelligent autonomous software engineering platform capable of performing software development tasks with minimal human intervention. Unlike traditional AI coding assistants, which only generate code based on user prompts, the proposed system executes the complete software development lifecycle automatically.

The system begins by accepting a software issue from GitHub or a textual task description. A Planner Agent analyzes the task and divides it into structured subtasks. These subtasks are forwarded to the Coder Agent, which generates source code inside an isolated Docker environment. Once the implementation is completed, the Tester Agent automatically executes the project's test suite and validates the generated solution.

An independent Critic Agent then reviews the generated code, evaluates code quality, checks compliance with coding standards, and determines whether the implementation should be accepted or rejected. If the implementation fails testing or review, the workflow automatically returns to the Coder Agent for self-correction. This retry loop continues until the code satisfies all validation criteria or the retry limit is reached.

Finally, an Escalation Agent intervenes whenever the confidence score falls below a predefined threshold or repeated failures occur. The system generates a structured failure report and notifies the developer instead of executing unsafe code.

4.2 System Modules

The proposed system consists of the following major modules:

- User Management Module
- Planner Agent Module
- Coder Agent Module
- Tester Agent Module
- Critic Agent Module
- Escalation Agent Module
- Dashboard Module
- GitHub Integration Module
- Database Module
- Logging Module
- Authentication Module

4.3 User Management Module

This module manages authorized users of the platform.

Major functionalities include:

- User Registration
- Secure Login
- JWT Authentication

- User Profile Management
- Password Encryption
- Role Management
- Session Handling

Only authenticated users are allowed to execute software engineering tasks through the platform.

4.4 Planner Agent Module

The Planner Agent acts as the coordinator of the entire system.

Major responsibilities include:

- Reading GitHub Issues
- Understanding user requirements
- Breaking large tasks into subtasks
- Generating execution plans
- Assigning work to the Coder Agent
- Managing workflow sequence

The Planner Agent reduces complexity by converting unstructured software requirements into organized execution plans.

4.5 Coder Agent Module

The Coder Agent is responsible for software implementation.

Its major functionalities include:

- Reading execution plans
- Understanding project structure
- Generating source code
- Modifying existing files
- Writing documentation
- Updating configuration files
- Creating Git commits

The generated code is executed only inside an isolated Docker container to ensure complete system security.

4.6 Tester Agent Module

The Tester Agent validates the generated implementation.

Major functions include:

- Unit Testing
- Integration Testing

- Build Validation
- Error Detection
- Test Report Generation
- Performance Evaluation

If test cases fail, detailed reports are generated and returned to the Coder Agent for correction.

4.7 Critic Agent Module

The Critic Agent performs independent software review.

Major functionalities include:

- Code Review
- Coding Standard Validation
- Performance Analysis
- Security Review
- Detecting Logical Errors
- Reviewing Test Results

Unlike ordinary AI systems, the Critic Agent independently evaluates the generated implementation before approval.

4.8 Escalation Agent

The Escalation Agent serves as the final safety mechanism.

Major functions include:

- Monitoring retry count
- Evaluating confidence score
- Blocking unsafe execution
- Generating failure reports
- Sending human notifications

If repeated failures occur, execution stops automatically to avoid deploying incorrect software.

4.9 Dashboard Module

The dashboard provides real-time monitoring of all AI agents.

The dashboard displays:

- Running Agent
- Completed Tasks
- Failed Tasks
- Retry Count
- Execution Time

- Resource Usage
- Confidence Score
- Test Results
- Logs
- Pull Request Status

This dashboard improves transparency by allowing developers to monitor every stage of execution.

4.10 GitHub Integration Module

This module communicates directly with GitHub repositories.

Major functionalities include:

- Reading Issues
- Creating Branches
- Updating Files
- Creating Pull Requests
- Fetching Repository Information
- Commit Management

CHAPTER 5

Methodology, Architecture, Database, Security and Requirements

5.1 Development Methodology

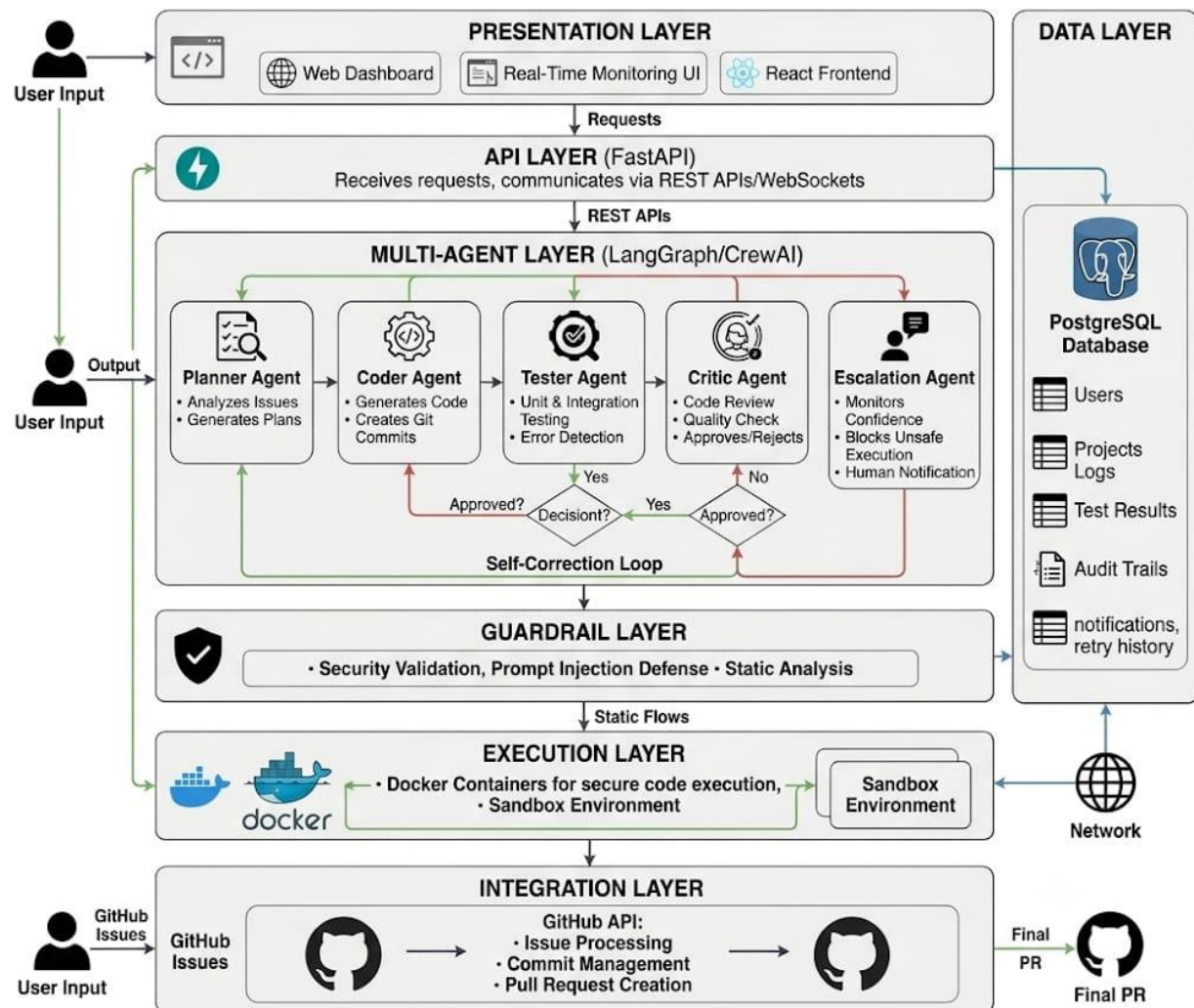
The proposed system follows the Agile Software Development Methodology.

Development phases include:

- Requirement Analysis
- System Design
- Agent Development
- Backend Development
- Dashboard Development
- Database Design
- Integration
- Testing
- Deployment

Each module is developed independently before integration into the complete system.

PROJECT ARCHITECTURE DIAGRAM: AUTONOMOUS OPS/DEV AGENT WITH REAL GUARDRAILS



5.2 Proposed System Architecture

The architecture consists of six layers.

- Presentation Layer**

Provides Web Dashboard and User Interface.

- API Layer**

Developed using FastAPI to receive requests and communicate between modules.

- **Multi-Agent Layer**

Contains:

- Planner Agent
- Coder Agent
- Tester Agent
- Critic Agent
- Escalation Agent

Guardrail Layer

Validates outputs before execution.

- **Execution Layer**

Uses Docker containers to safely execute generated code.

- **Data Layer**

Stores users, execution logs, tasks, reports, and GitHub activity using PostgreSQL.

5.3 Database Design

The proposed database consists of the following tables:

- Users
- Projects
- Tasks
- Agent Logs
- Test Results
- Pull Requests
- Docker Sessions
- Audit Logs
- Notifications
- Retry History

The relational database maintains complete execution history for every software task.

5.4 Security Methodology

Security is one of the major objectives of this project.

Security features include:

- JWT Authentication
- Password Hashing
- Docker Isolation

- API Authentication
- HTTPS Communication
- Input Validation
- Role-Based Access Control
- Guardrail Validation
- SQL Injection Prevention
- Audit Logging

These mechanisms ensure safe execution of AI-generated software.

5.5 Testing Methodology

Different testing techniques will be adopted.

- **Unit Testing**

Individual modules will be tested independently.

- **Integration Testing**

Communication among multiple agents will be validated.

- **Functional Testing**

Complete workflow testing.

- **Security Testing**

Authentication and authorization testing.

- **Performance Testing**

Execution speed and scalability analysis.

- **User Acceptance Testing**

Final validation before deployment.

5.6 Software Requirements

Operating System

- Windows 10/11
- Linux

Programming Language

- Python

Backend Framework

- FastAPI

Frontend

- HTML
- CSS
- JavaScript
- React

Database

- PostgreSQL

Containerization

- Docker

AI Framework

- LangGraph
- CrewAI

Version Control

- GitHub

IDE

- Visual Studio Code

Browser

- Google Chrome

5.7 Hardware Requirements

Minimum Requirements

- Intel Core i3 Processor
- 8 GB RAM
- 20 GB Storage
- Stable Internet Connection

Recommended Requirements

- Intel Core i5/i7
- 16 GB RAM
- SSD Storage
- Dedicated Internet Connection

CHAPTER 6

Expected Results

After successful implementation, the proposed system is expected to provide:

- Autonomous issue processing
- Automatic software planning
- Intelligent code generation
- Automatic testing
- Independent code review
- Self-correction mechanism
- Secure execution
- Live monitoring dashboard
- GitHub Pull Request generation
- Complete audit logs

The system is expected to reduce software development time while improving software quality and reliability.

Discussion

The proposed platform combines modern AI technologies with DevOps automation to build a fully autonomous software engineering system. By integrating intelligent planning, secure execution, automatic testing, and self-correction, the project minimizes repetitive development tasks and enables developers to focus on solving complex business problems.

CHAPTER 7

Conclusion

The proposed **Autonomous Ops/Dev Agent with Real Guardrails** introduces a next-generation AI-driven software engineering platform capable of performing software development tasks with minimal human intervention.

The project integrates multiple specialized AI agents that collaboratively perform planning, coding, testing, reviewing, and decision-making. Unlike conventional AI coding assistants, the proposed system validates every software change before approval, making the development process more reliable and secure.

The use of Docker containers, FastAPI, LangGraph, CrewAI, PostgreSQL, and GitHub APIs provides a scalable and production-ready architecture. The inclusion of real-time monitoring and execution guardrails ensures transparency and operational safety.

Overall, the project demonstrates how multi-agent AI systems can transform modern DevOps workflows into intelligent, autonomous, and self-correcting software engineering platforms.

Future Scope

The project can be enhanced with the following features:

- Cloud Deployment on AWS, Azure, or GCP
- Kubernetes Integration
- Multi-Repository Support
- Voice-Based Issue Submission
- AI-Based Code Optimization
- Automatic Documentation Generation
- Security Vulnerability Detection
- Continuous Deployment (CI/CD)
- Multi-Language Programming Support
- Cost Optimization Analytics
- Mobile Monitoring Application
- Team Collaboration Features
- Enterprise DevOps Integration
- Reinforcement Learning-Based Agent Improvement
- Predictive Failure Analysis

These enhancements can transform the project into a production-grade autonomous software engineering platform suitable for enterprise software development.

REFERENCES

1. Python Software Foundation, *Python Documentation*.
2. FastAPI Official Documentation.
3. CrewAI Documentation.
4. LangGraph Documentation.
5. Docker Official Documentation.
6. PostgreSQL Documentation.
7. GitHub REST API Documentation.
8. Guardrails AI Documentation.
9. Ian Sommerville, *Software Engineering*, Pearson.
10. Roger S. Pressman & Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

